

```

ankur@ankur-Laptop:~/kernel/modules$ make
make -C /lib/modules/2.6.34.12/build M=/home/ankur/kernel/modules modules
make[1]: Entering directory '/home/ankur/kernel/linux-2.6.34.12'
Building modules, stage 2.
  MODPOST 1 modules
make[1]: Leaving directory '/home/ankur/kernel/linux-2.6.34.12'
ankur@ankur-Laptop:~/kernel/modules$ sudo insmod hello.ko
ankur@ankur-Laptop:~/kernel/modules$ sudo rmmod hello.ko
ankur@ankur-Laptop:~/kernel/modules$ dmesg | tail -2
[34429.922294] Hello world
[34429.922294] Goodbye

```

Figure 1: Build, load, remove and verify output

the *module.h* header file. It is also mandatory if you are writing a module for the kernel.

3. *MODULE_LICENSE()* will specify the licence type under which this module is written. Get into a habit of writing it, otherwise you will unnecessary 'taint' warnings in the kernel log. I have used the GPL.
4. A *static int* function is created to execute on loading of the module. This function is also known as an initialise function. It must not be of the *void* type because the function must return false if initialisation failed.
5. There is no *printf()* in kernel programming; whereas here, there is *printk()*, which copies the formatted string into the kernel log buffer, which is normally read by the *syslog* program. It works similar to *printf()*; the only difference is that *printk()* enables us to specify a priority flag that is used by *syslogd* to decide where to display kernel messages. In the line, *KERN_ALERT*, is the flag. *KERN_ERR*, *KERN_WARNING* etc, are some other flags. It also works well without the flags.
6. Like the initialise function, you made a function which will be executed when you remove the module from the kernel space. The function usually has a *void* return type.
7. The *module_init(function name)* call will tell the kernel which function to execute on the module's initialisation.
8. The *module_exit(function name)* call will tell the kernel which function to execute on module removal.

Makefile

```

obj-m += hello.o #1

all:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules #2

clean:
    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean

```

Let's compile the above module using a *Makefile* with the above code, which should be in the same folder as *hello.c*; otherwise, you need to specify an absolute path to the file. *obj-m* is a list of what kernel modules to build. The *.o* and other objects will be automatically built from

the corresponding *.c* file (no need to list the source files explicitly). So just give the name of your program file against *obj-m* (without the *.c* extension).

Describing the base build path

It will compile and build your module with respect to the mentioned path. Please note that the above path is generic for most Linux distributions. It will act as a base for the module build.

Compile, load and remove

Compile the module using *make*. A lot of files will be generated, such as *hello.ko*, *hello.mod.o*, *hello.o*, etc. Load the module into kernel space using the *insmod* utility, and you must need *root* permissions to do that. So run *sudo insmod hello.ko* in the terminal. Your module gets loaded. To check the output of the module, since *printk()* doesn't write to the console (though this can be done by a kernel hack), check the kernel log with the *dmesg* command. Run it and you will see 'Hello World' in the output.

Now let's remove the module from the kernel space using *sudo rmmod hello.ko* in the terminal. Verify this using *dmesg* again; you will see *GoodBye* printed at the end of the *syslog* stack. See the screen-shot (Figure 1).



Note: You can't load a module twice successively. If you have just loaded a module, and you want to load it again, you need to unload it first; else you will get a console error message.

Passing parameters

If you want to pass a single or a series of parameters along with the module itself, which is quite useful when your module depends on other modules' output, follow the steps shown below.

Passing a single parameter

Module filename: *parameter.c*

```

#include<linux/init.h>
#include<linux/module.h>
#include<linux/moduleparam.h> #1

MODULE_LICENSE("GPL");

int paramTest;
module_param(paramTest, int, S_IRUSR|S_IWUSR); #2

static int param_init(void) {
    printk(KERN_ALERT "Showing the parameter Demo");
    printk(KERN_ALERT "Value of paramTest is: %d", paramTest); #3
    return 0;
}

```